torch.fx.subgraph_rewriter.replace_pattern

torch.fx.subgraph_rewriter.replace_pattern(*gm*, *pattern*, *replacement*) # [source]

Matches all possible non-overlapping sets of operators and their data dependencies (`pattern`) in the Graph of a GraphModule (`gm`), then replaces each of these matched subgraphs with another subgraph (`replacement`).

Parameters:

- **gm** (*GraphModule*) – The GraphModule that wraps the Graph to operate on
- **pattern** (*Callable[[...], Any]* | *GraphModule*) – The subgraph to match in `gm` for replacement
- **replacement** (*Callable[[...], Any]* | *GraphModule*) – The subgraph to replace `pattern` with

Returns:

A list of `Match` objects representing the places in the original graph that `pattern` was matched to. The list is empty if there are no matches. `Match` is defined as:

```
class Match(NamedTuple):
    # Node from which the match was found
    anchor: Node
    # Maps nodes in the pattern subgraph to nodes in the larger graph
    nodes_map: Dict[Node, Node]
```

Return type:

List[Match]

Examples:

Ask AI

```
import torch
from torch.fx import symbolic_trace, subgraph_rewriter


class M(torch.nn.Module):
    def __init__(self) -> None:
        super().__init__()

    def forward(self, x, w1, w2):
        m1 = torch.cat([w1, w2]).sum()
        m2 = torch.cat([w1, w2]).sum()
        return x + torch.max(m1) + torch.max(m2)


def pattern(w1, w2):
    return torch.cat([w1, w2])


def replacement(w1, w2):
    return torch.stack([w1, w2])


traced_module = symbolic_trace(M())

subgraph_rewriter.replace_pattern(traced_module, pattern, replacement)
```

The above code will first match `pattern` in the `forward` method of `traced_module`. Pattern-matching is done based on use-def relationships, not node names. For example, if you had `p = torch.cat([a, b])` in `pattern`, you could match `m = torch.cat([a, b])` in the original `forward` function, despite the variable names being different (`p` vs `m`).

The `return` statement in `pattern` is matched based on its value only; it may or may not match to the `return` statement in the larger graph. In other words, the pattern doesn't have to extend to the end of the larger graph.

When the pattern is matched, it will be removed from the larger function and replaced by `replacement`. If there are multiple matches for `pattern` in the larger function, each non-overlapping match will be replaced. In the case of a match overlap, the first found match in the set of overlapping matches will be replaced. ("First" here being defined as the first in a topological ordering of the Nodes' use-def relationships. In most cases, the first Node is the parameter that appears directly a he last Node is whatever the function returns.)

Ask AI

One important thing to note is that the parameters of the `pattern` Callable must be used in the Callable itself, and the parameters of the `replacement` Callable must match the pattern. The first rule is why, in the above code block, the `forward` function has parameters `x, w1, w2`, but the `pattern` function only has parameters `w1, w2`. `pattern` doesn't use `x`, so it shouldn't specify `x` as a parameter. As an example of the second rule, consider replacing

```
def pattern(x, y):
    return torch.neg(x) + torch.relu(y)
```

with

```
def replacement(x, y):
    return torch.relu(x)
```

In this case, `replacement` needs the same number of parameters as `pattern` (both `x` and `y`), even though the parameter `y` isn't used in `replacement`.

After calling `subgraph_rewriter.replace_pattern`, the generated Python code looks like this:

```
def forward(self, x, w1, w2):
    stack_1 = torch.stack([w1, w2])
    sum_1 = stack_1.sum()
    stack_2 = torch.stack([w1, w2])
    sum_2 = stack_2.sum()
    max_1 = torch.max(sum_1)
    add_1 = x + max_1
    max_2 = torch.max(sum_2)
    add_2 = add_1 + max_2
    return add_2
```

Send Feedback

Note

Backwards-compatibility for this API is guaranteed.

NodeSourceAction torch.fx.subgraph_rewriter.replace_pattern_with_filters

Built with the PyData Sphinx Theme 0.15.4.

Ask AI

Docs

Access comprehensive developer documentation for PyTorch

View Docs

Tutorials

Get in-depth tutorials for beginners and advanced developers

View Tutorials

Resources

Find development resources and get your questions answered

View Resources

Stay in touch for updates, event info, and the latest news

First Name* Last Name* Email*

Select Country* SUBMIT

By submitting this form, I consent to receive marketing emails from the LF and its projects regarding their events, training, research, developments, and related announcements. I understand that I can unsubscribe at any time using the links in the footers of the emails I receive. Privacy Policy.

© PyTorch. Copyright © The Linux Foundation®. All rights reserved. The Linux Foundation has registered trademarks and uses trademarks. For more information, including terms of use, privacy policy, and trademark usage, please see our Policies page. Trademark Usag

Ask AI